Actividad Práctica 02

Fundamentos de Databricks — Blur Consulting

Estudiante: Isabela Tellez

Fecha: 30 de octubre de 2026

Contenido

Introducción 3
Objetivo 3
Justificación 3
Metodología 3
Desarrollo 4
Preparación del Entorno 4
Código Ejecutado 4
Captura de código y salida por consola ..... **¡Error! Marcador no definido.**
Ejercicio 1: Pedidos por estado 5
Código Ejecutado 5
Ejercicio 2: Evolución anual 6
Código Ejecutado 6
Ejercicio 3: Pedidos grandes ya finalizados 7
Código Ejecutado 7
Ejercicio 4: Clientes por segmento 8
Código Ejecutado 8
Ejercicio 5: Clientes por país 9
Código Ejecutado 9
Ejercicio 6: Ticket promedio por segmento 10
Código Ejecutado 10

Introducción

Objetivo

El objetivo de esta actividad es poner en práctica los conocimientos adquiridos sobre el uso de PySpark en Databricks, mediante el análisis y procesamiento de datos utilizando diferentes operaciones sobre DataFrames.

A través de las tablas de pedidos, clientes y países, se busca obtener información relevante mediante agrupaciones, filtros, agregaciones y uniones de datos. De esta manera, se pretende comprender cómo utilizar PySpark para consultar y analizar grandes volúmenes de información de forma eficiente.

Justificación

El uso de herramientas como PySpark permite trabajar con grandes cantidades de datos y obtener información útil para la toma de decisiones. Por ello, esta actividad permite familiarizarse con operaciones habituales en el análisis de datos, como calcular totales, promedios, contar registros y relacionar información procedente de distintas tablas.

Además, trabajar con Databricks facilita la ejecución de código PySpark en un entorno preparado para el procesamiento distribuido, reforzando los conocimientos prácticos necesarios para desarrollar tareas de análisis y tratamiento de datos.

Metodología

Para realizar esta actividad se ha utilizado Databricks Free Edition como entorno de trabajo y PySpark como herramienta para procesar y analizar los datos.

El procedimiento seguido ha sido el siguiente:

1. Carga de datos: se han importado las tablas orders, customer y nation del conjunto de datos samples.tpch.

2. Exploración y preparación: se han identificado las columnas necesarias para resolver cada uno de los ejercicios.

3. Procesamiento: se han aplicado operaciones de agrupación (groupBy), filtrado (filter), agregación (count, sum y avg), unión de tablas (join) y ordenación (orderBy).

4. Obtención de resultados: se han ejecutado las consultas para obtener los resultados solicitados en los seis ejercicios.

5. Documentación: se han incorporado los fragmentos de código y se han reservado espacios para añadir las capturas de pantalla de las ejecuciones realizadas en Databricks.

Desarrollo

Preparación del Entorno

Se importa el módulo de funciones de PySpark y se cargan las tablas orders, customer y nation, que se utilizarán en los ejercicios.

Código Ejecutado

```
from pyspark.sql import functions as F

orders = spark.table("samples.tpch.orders")

customer = spark.table("samples.tpch.customer")

nation = spark.table("samples.tpch.nation")
```



Ejercicio 1: Pedidos por estado

Enunciado: ¿Cuántos pedidos hay en cada estado (o_orderstatus) y cuál es el monto total de cada uno?

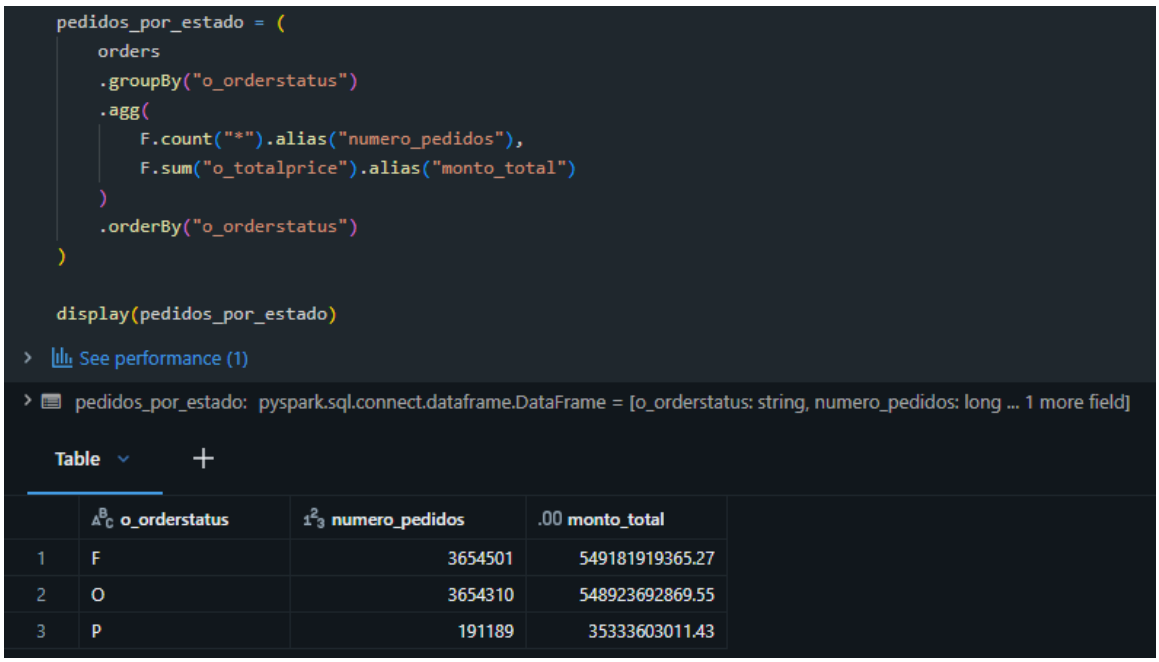
Desarrollo: Se agrupan los pedidos por estado y se calculan el número de pedidos y la suma de sus importes.

Código Ejecutado

```
pedidos_por_estado = (
orders
.groupBy("o_orderstatus")
.agg(
F.count("*").alias("numero_pedidos"),
F.sum("o_totalprice").alias("monto_total")
)
.orderBy("o_orderstatus")
)

display(pedidos_por_estado)
```

Resultado obtenido: Tabla con el estado del pedido, el número de pedidos y el monto total de cada estado.



Ejercicio 2: Evolución anual

Enunciado: ¿Cuántos pedidos y qué monto total hay por año (o_orderdate)? Ordenar del año más antiguo al más reciente.

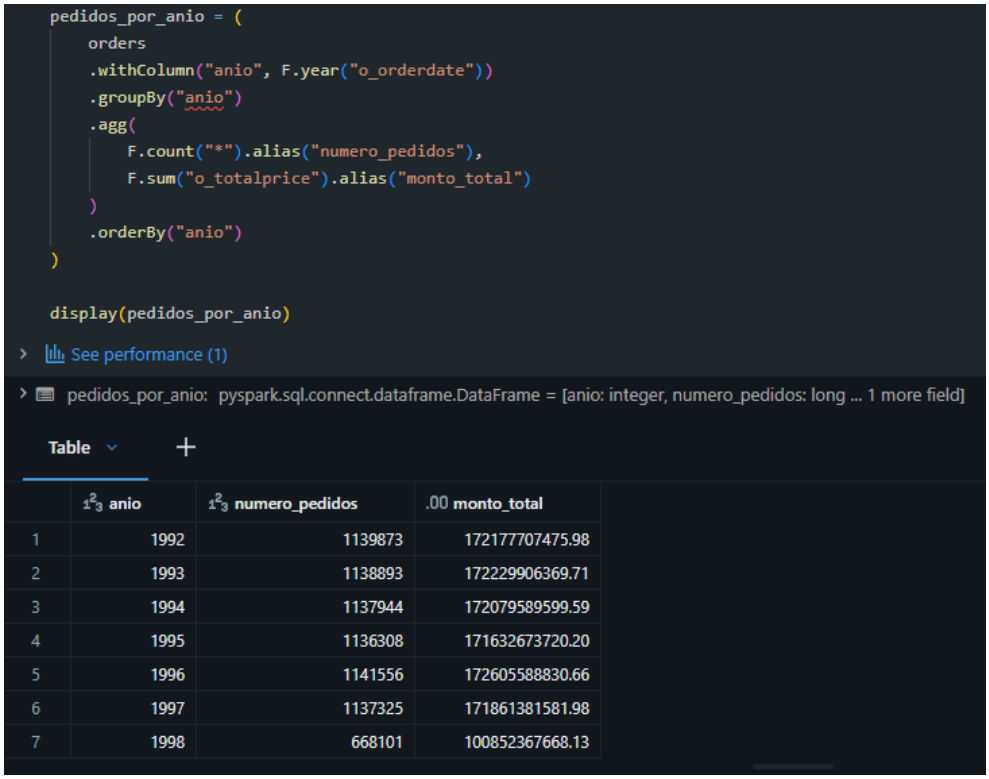
Desarrollo: Se extrae el año de la fecha de cada pedido, se agrupan los registros por año y se calculan el número de pedidos y el importe total.

Código Ejecutado

```
pedidos_por_anio = (
orders
.withColumn("anio", F.year("o_orderdate"))
.groupBy("anio")
.agg(
F.count("*").alias("numero_pedidos"),
F.sum("o_totalprice").alias("monto_total")
)
.orderBy("anio")
)

display(pedidos_por_anio)
```

Resultado obtenido: Tabla con el año, el número de pedidos y el monto total anual, ordenada cronológicamente.



	anio	numero_pedidos	monto_total
1	1992	1139873	172177707475.98
2	1993	1138893	172229906369.71
3	1994	1137944	172079589599.59
4	1995	1136308	171632673720.20
5	1996	1141556	172605588830.66
6	1997	1137325	171861381581.98
7	1998	668101	100852367668.13

Ejercicio 3: Pedidos grandes ya finalizados

Enunciado: ¿Cuántos pedidos finalizados (o_orderstatus = 'F') tienen un importe (o_totalprice) superior a 100.000?

Desarrollo: Se filtran los pedidos que cumplen simultáneamente ambas condiciones y se cuenta el número de registros resultantes.

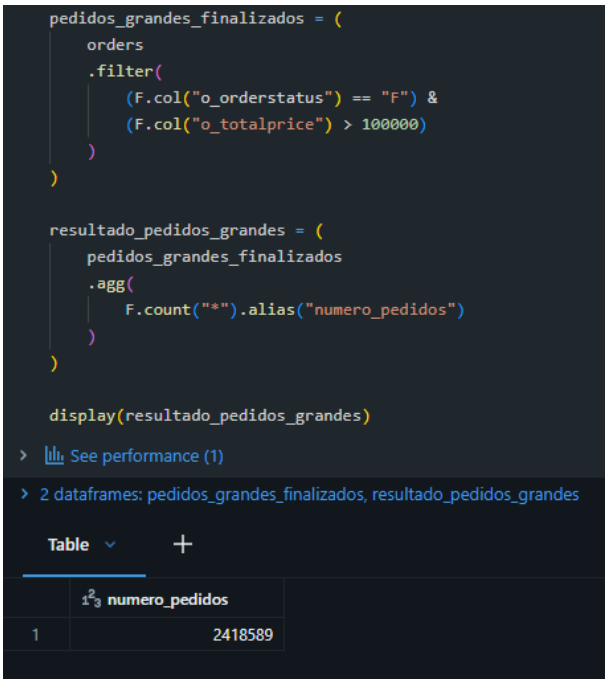
Código Ejecutado

```
pedidos_grandes_finalizados = (
orders
.filter(
(F.col("o_orderstatus") == "F") &
(F.col("o_totalprice") > 100000)
)
)

resultado_pedidos_grandes = (
pedidos_grandes_finalizados
.agg(F.count("*").alias("numero_pedidos"))
)

display(resultado_pedidos_grandes)
```

Resultado obtenido: Tabla con una única columna, numero_pedidos, que indica cuántos pedidos cumplen las condiciones.



Ejercicio 4: Clientes por segmento

Enunciado: ¿Cuántos clientes hay en cada segmento de mercado (c_mktsegment)? Ordenarlos de mayor a menor.

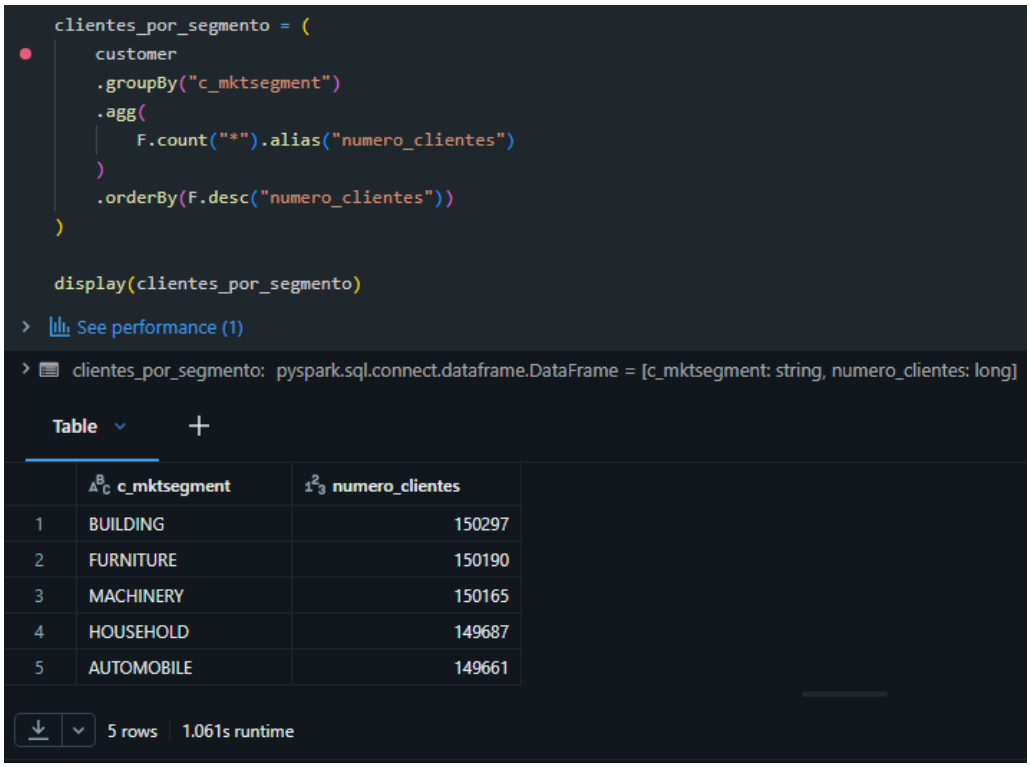
Desarrollo: Se agrupan los clientes por segmento de mercado, se cuenta el número de clientes de cada grupo y se ordenan los resultados de forma descendente.

Código Ejecutado

```
clientes_por_segmento = (
customer
.groupBy("c_mktsegment")
.agg(
F.count("*").alias("numero_clientes")
)
.orderBy(F.desc("numero_clientes"))
)

display(clientes_por_segmento)
```

Resultado obtenido: Tabla con cada segmento de mercado y su número de clientes, ordenada de mayor a menor.



Ejercicio 5: Clientes por país

Enunciado: ¿En qué cinco países (n_name) hay más clientes? Mostrar el país y el número de clientes, de mayor a menor.

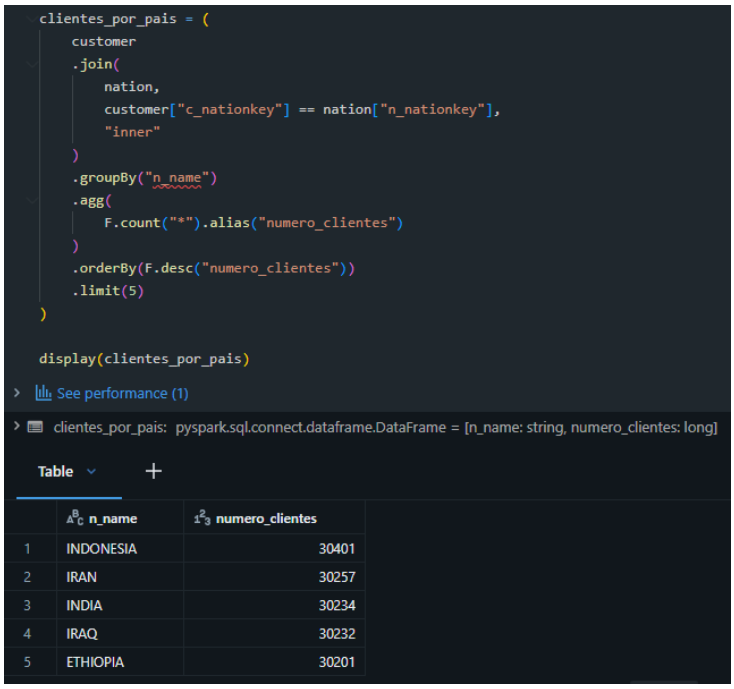
Desarrollo: Se unen las tablas customer y nation mediante sus claves. Después, se agrupan los clientes por país, se cuentan y se seleccionan los cinco países con más clientes.

Código Ejecutado

```
clientes_por_pais = (
customer
.join(
nation,
customer["c_nationkey"] == nation["n_nationkey"],
"inner"
)
.groupBy("n_name")
.agg(
F.count("*").alias("numero_clientes")
)
.orderBy(F.desc("numero_clientes"))
.limit(5)
)

display(clientes_por_pais)
```

Resultado obtenido: Tabla con cinco filas y dos columnas: país y número de clientes.



Ejercicio 6: Ticket promedio por segmento

Enunciado: ¿Cuál es el ticket promedio (monto promedio por pedido, o_totalprice) de cada segmento de mercado? Ordenarlo de mayor a menor.

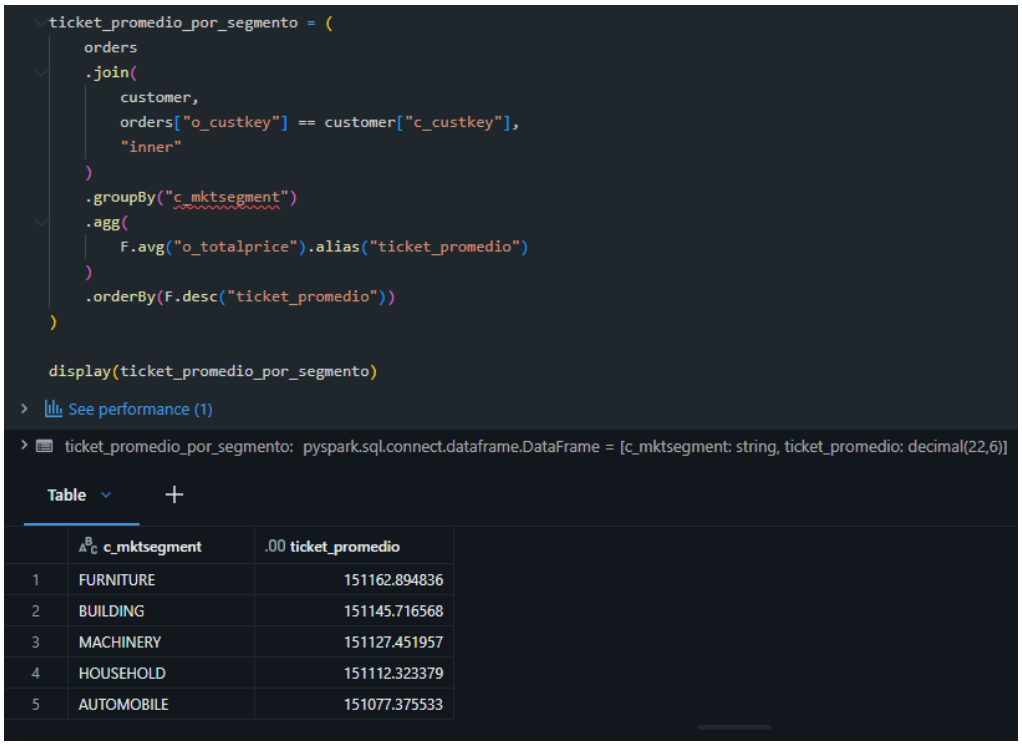
Desarrollo: Se unen los pedidos con sus clientes mediante sus claves. Luego, se agrupan por segmento de mercado y se calcula el importe promedio de los pedidos.

Código Ejecutado

```
ticket_promedio_por_segmento = (
orders
.join(
customer,
orders["o_custkey"] == customer["c_custkey"],
"inner"
)
.groupBy("c_mktsegment")
.agg(
F.avg("o_totalprice").alias("ticket_promedio")
)
.orderBy(F.desc("ticket_promedio"))
)

display(ticket_promedio_por_segmento)
```

Resultado obtenido: Tabla con cada segmento de mercado y su ticket promedio, ordenada de mayor a menor.



	c_mktsegment	ticket_promedio
1	FURNITURE	151162.894836
2	BUILDING	151145.716568
3	MACHINERY	151127.451957
4	HOUSEHOLD	151112.323379
5	AUTOMOBILE	151077.375533

Conclusiones

En esta actividad se han aplicado operaciones de PySpark para analizar los datos de pedidos y clientes en Databricks. Se han utilizado agrupaciones (groupBy), filtros (filter), agregaciones (count, sum y avg), uniones de tablas (join) y ordenación de resultados (orderBy).

Estas operaciones permiten obtener información sobre los pedidos, la evolución anual de los importes, los segmentos de mercado y la distribución de clientes por país. La práctica muestra cómo PySpark facilita el procesamiento y análisis distribuido de datos.